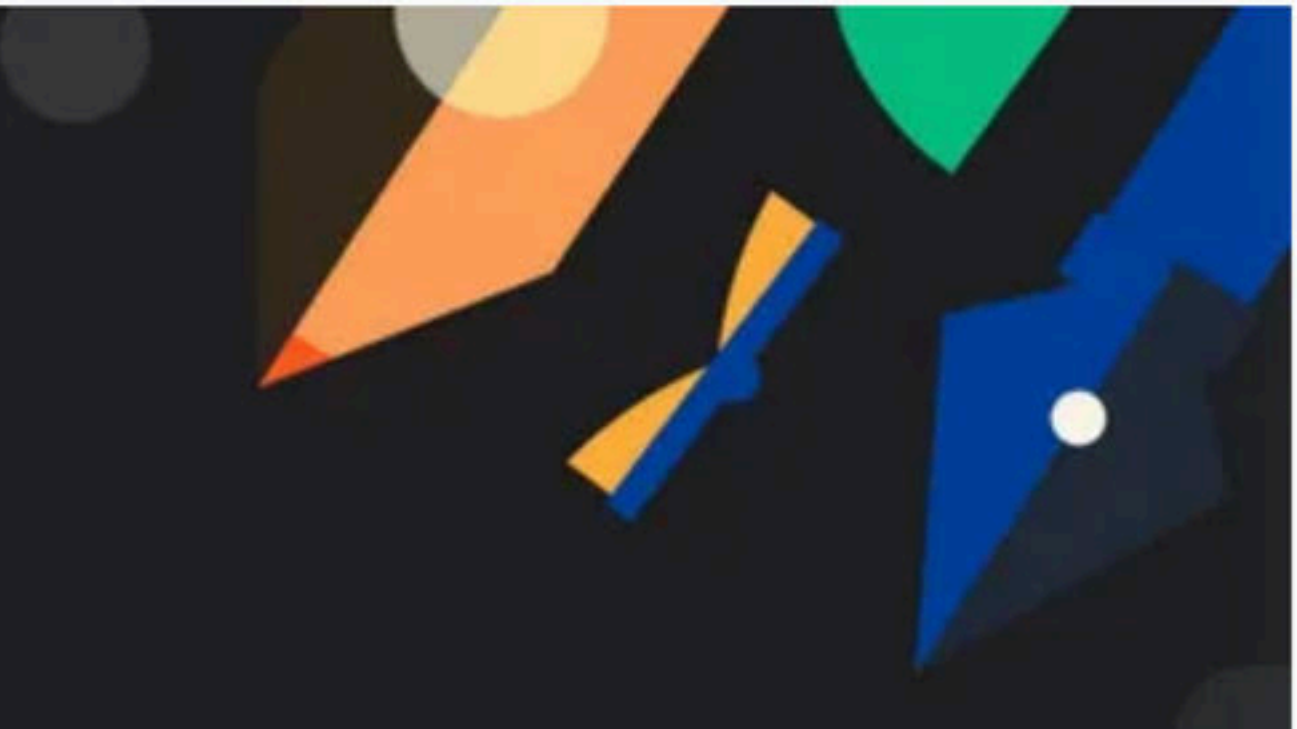




Graph Class - 1

Special class

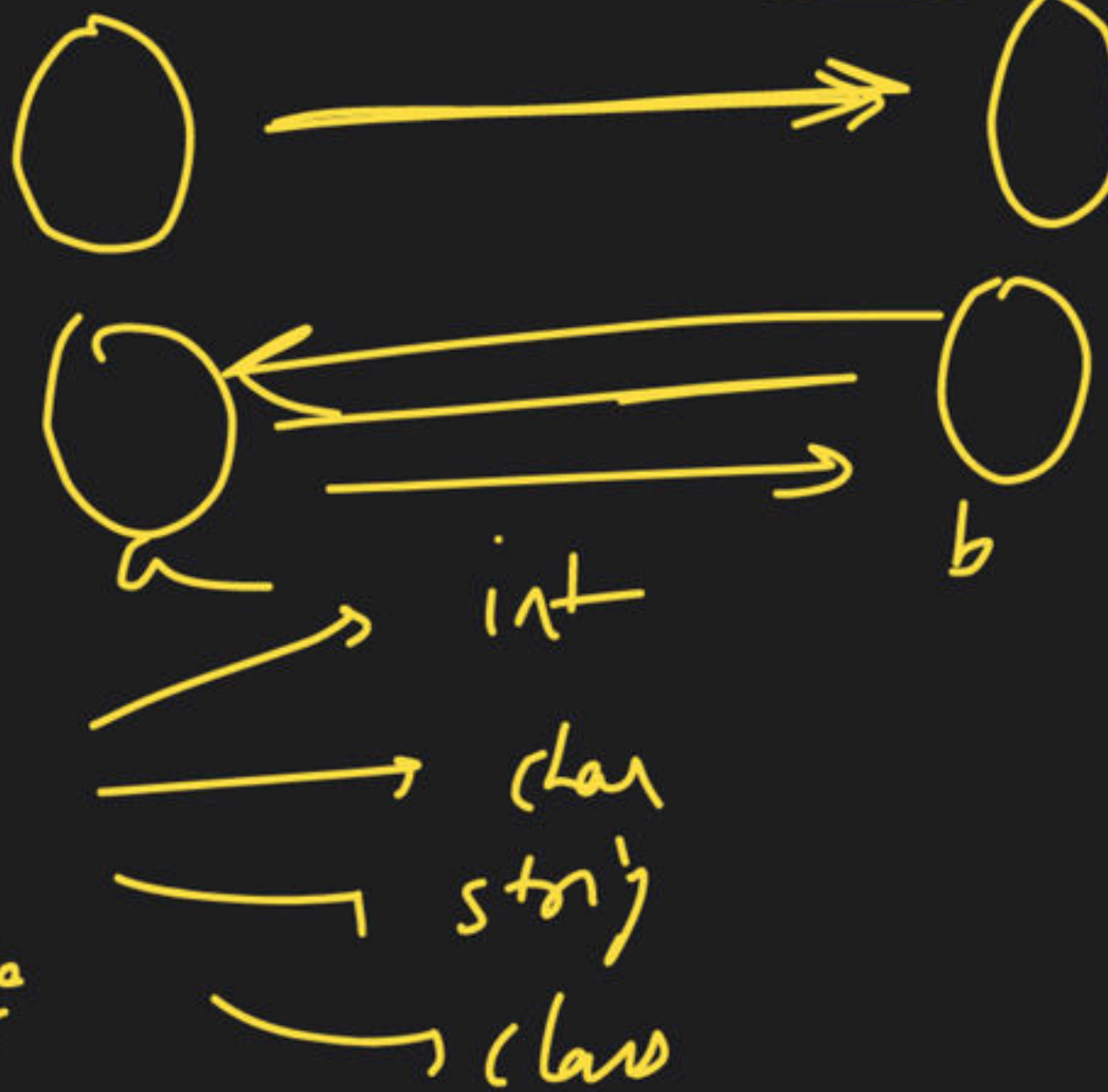
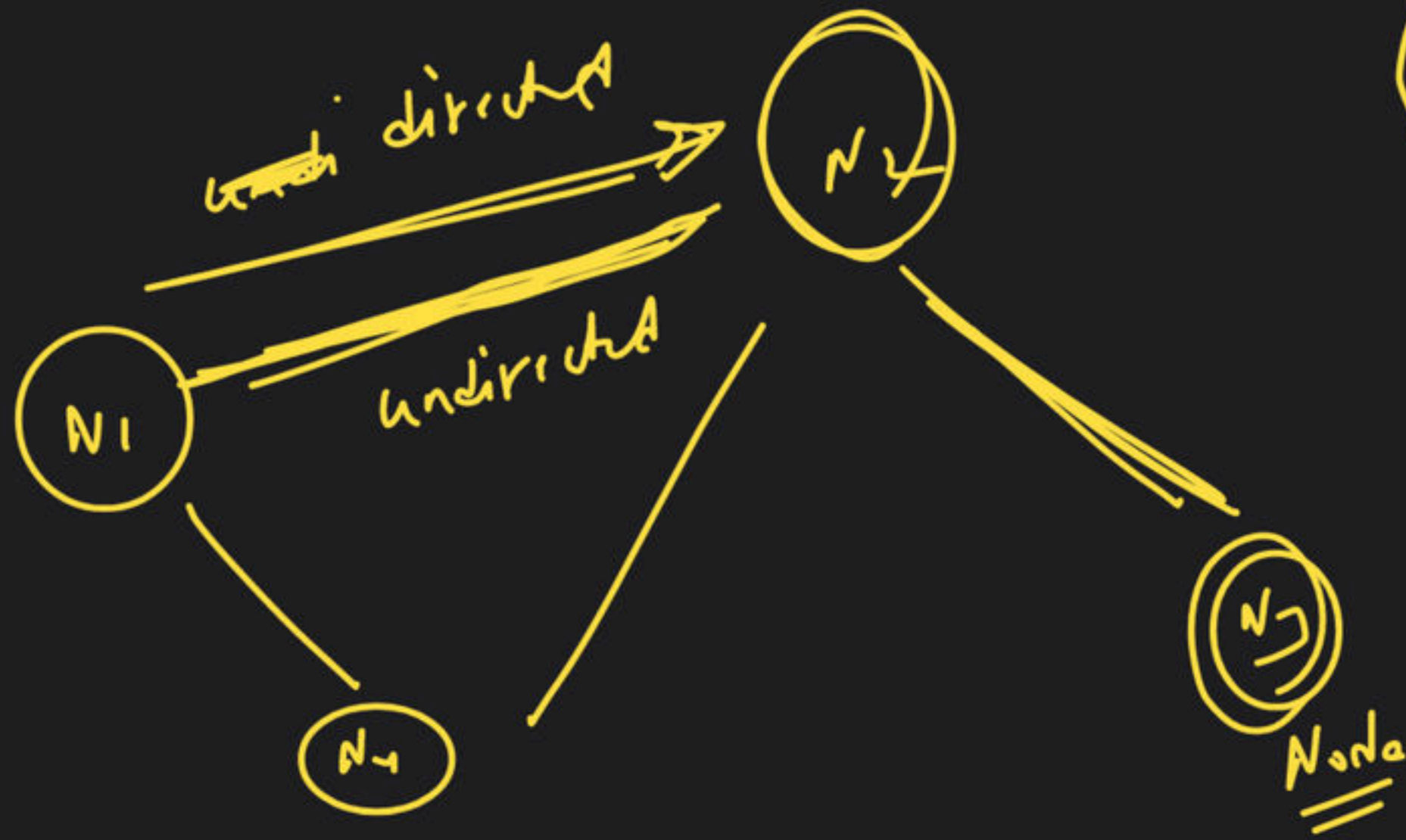
→ Graphs

(1) S

Edges

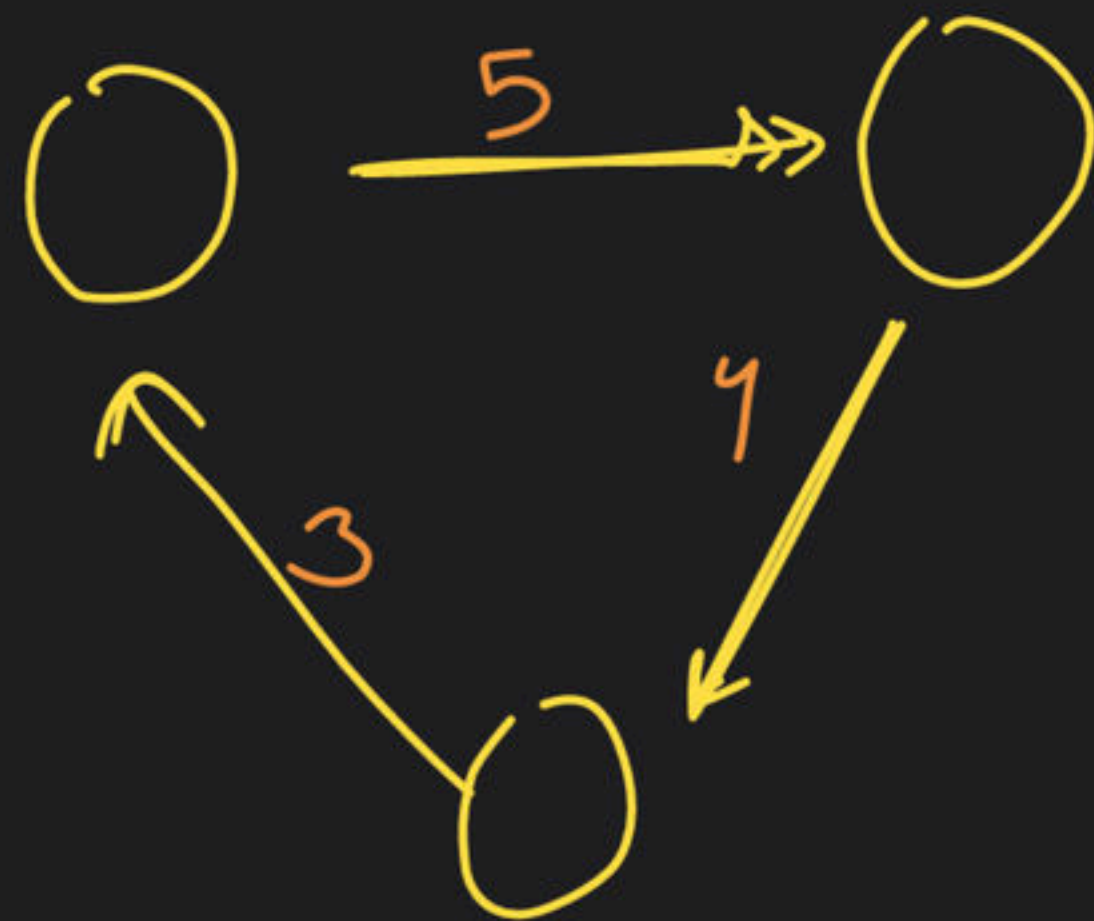
Nodus/Vertex

clone a graph
clone a tree
check whether
graph is a tree
or not



graph
edge → ~~directed~~
 → undirected
node

~~weighted~~ Directed Graph



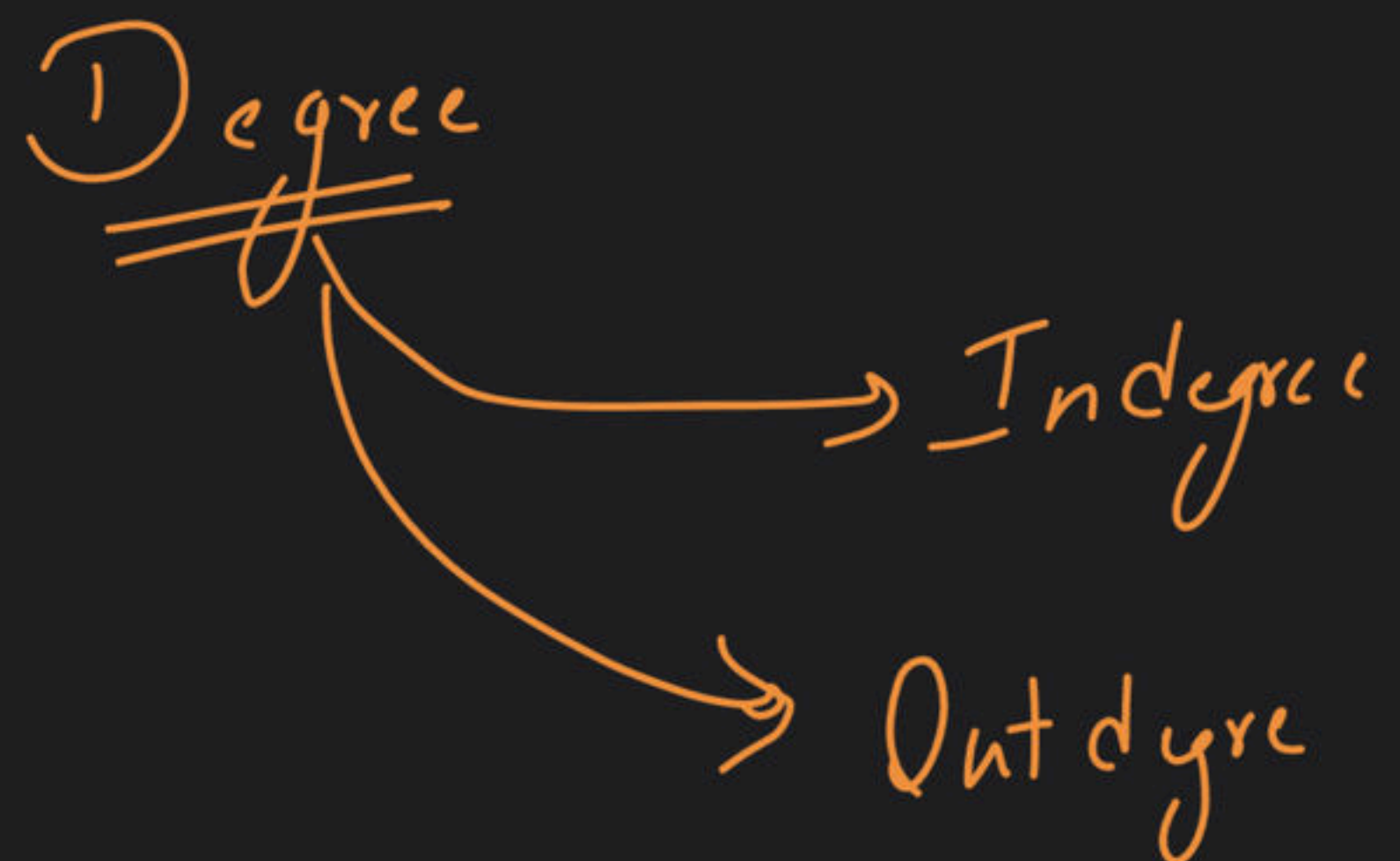
~~unweighted~~ undirected graph



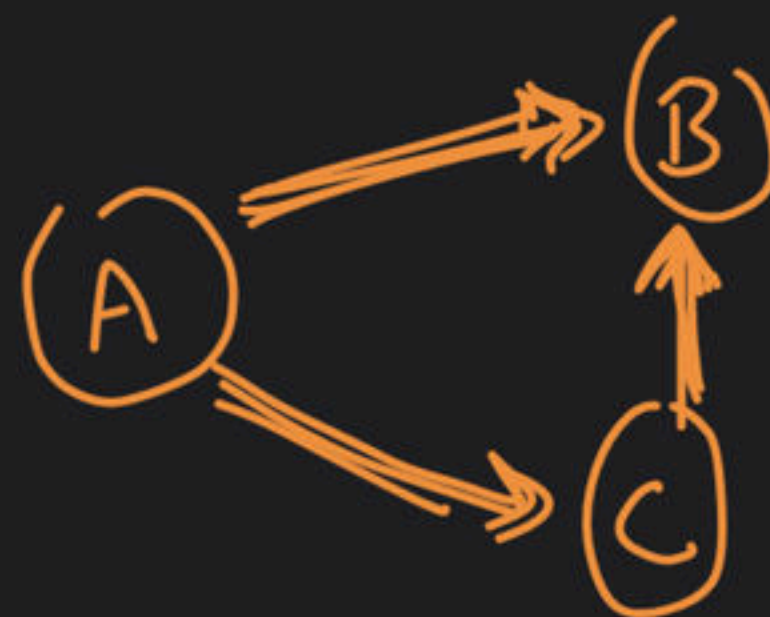
weighted
undirected
graph



Degree



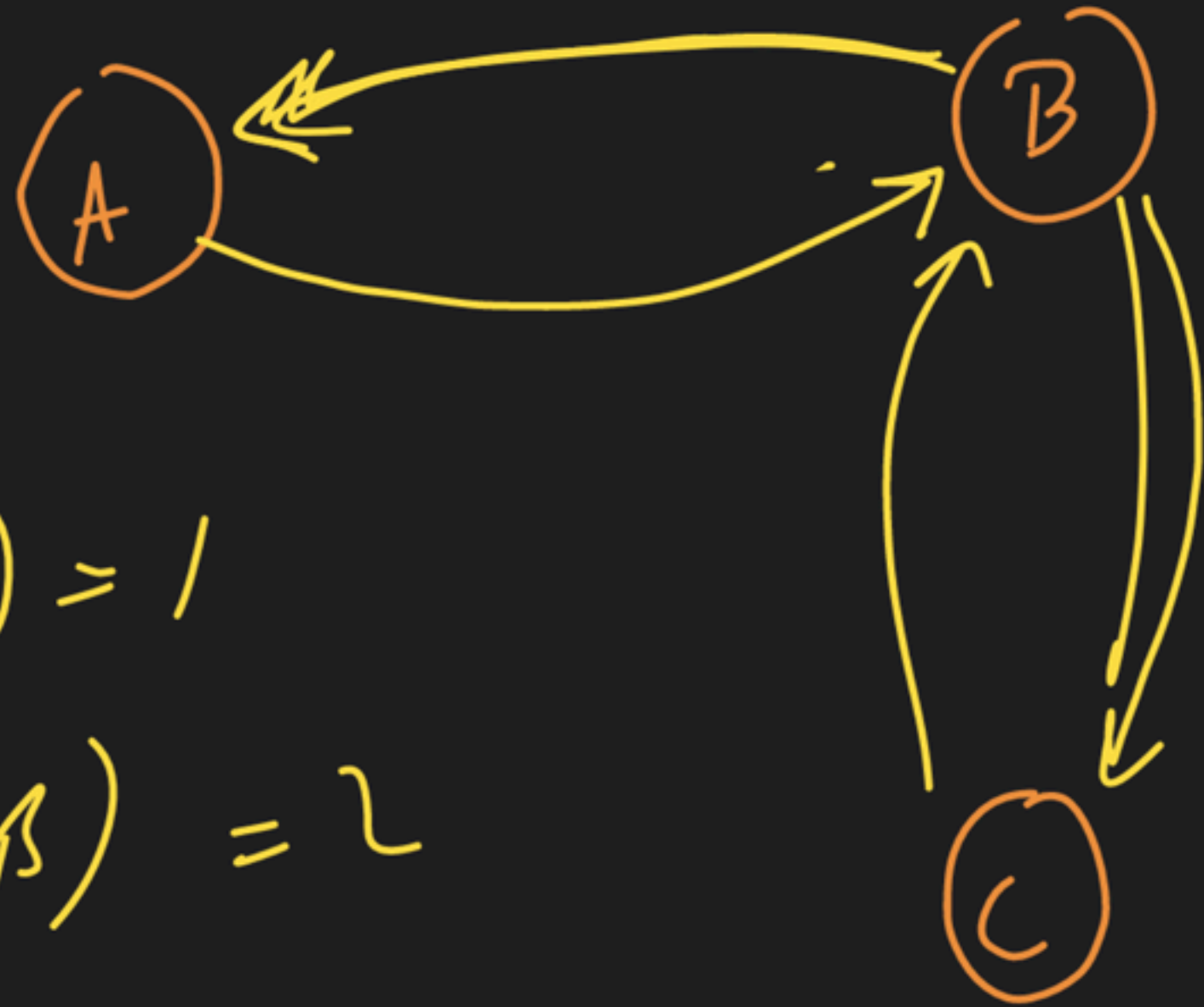
Directed



$$\text{Indegree}(A) = 0$$

$$\text{Outdyre}(A) = 2$$

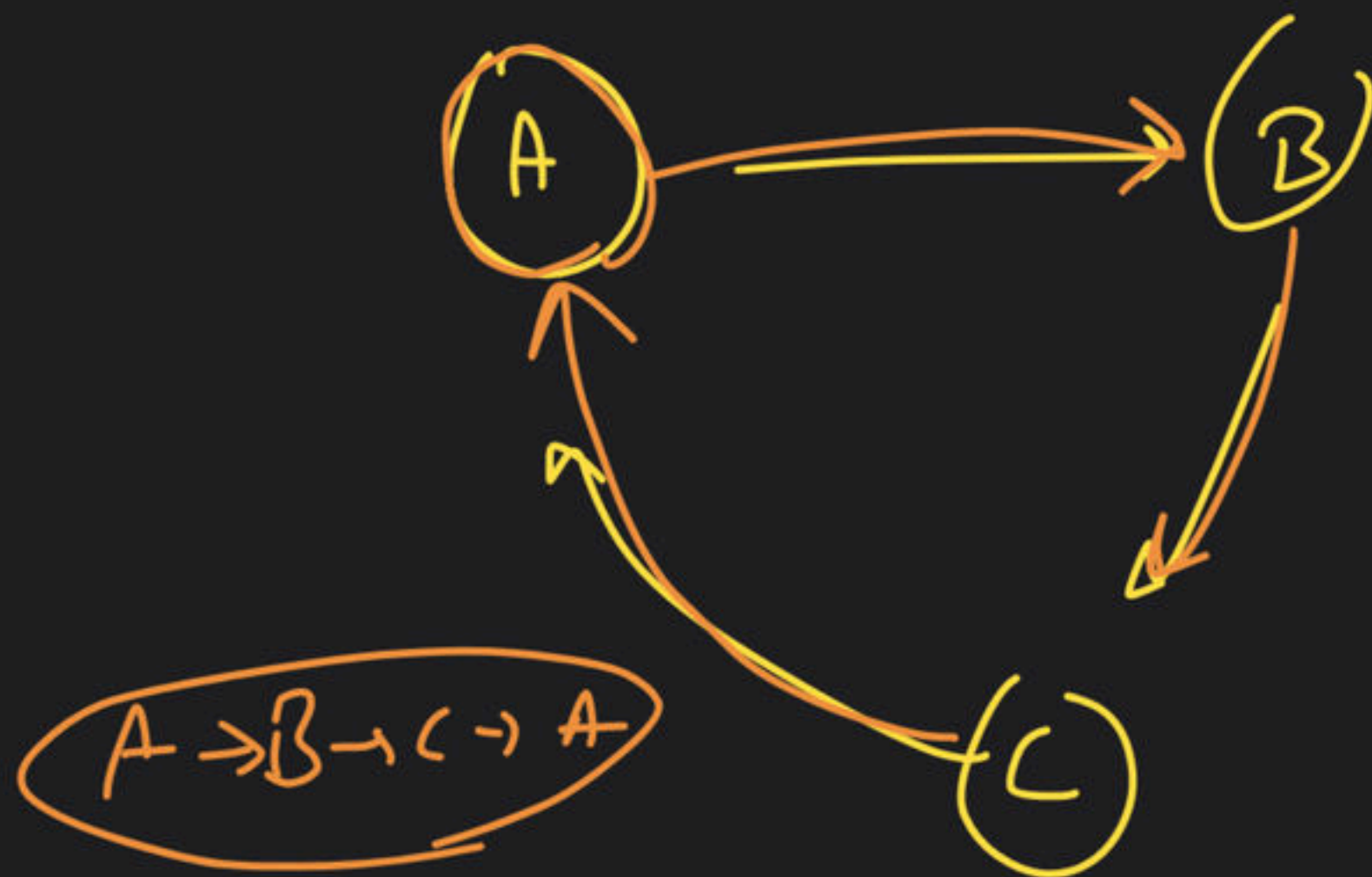
$$\text{Indegree}(B) = 2$$



$$\text{indegree}(A) = 1$$

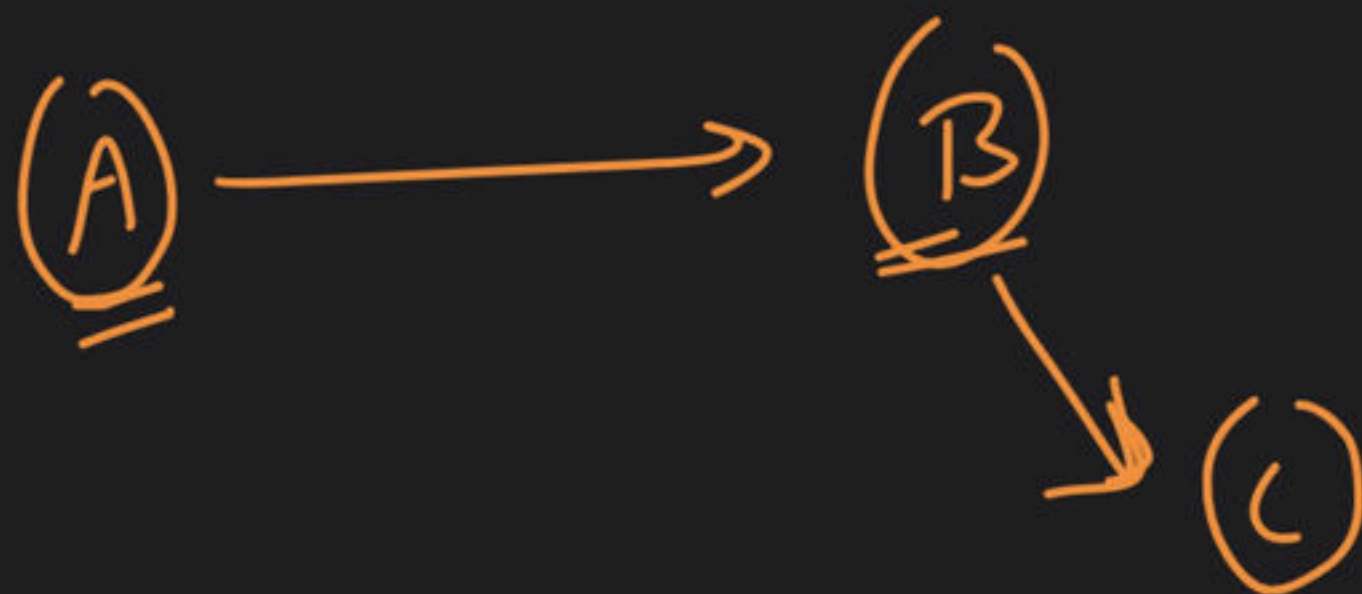
$$\text{outdegree}(B) = 2$$

Cycle



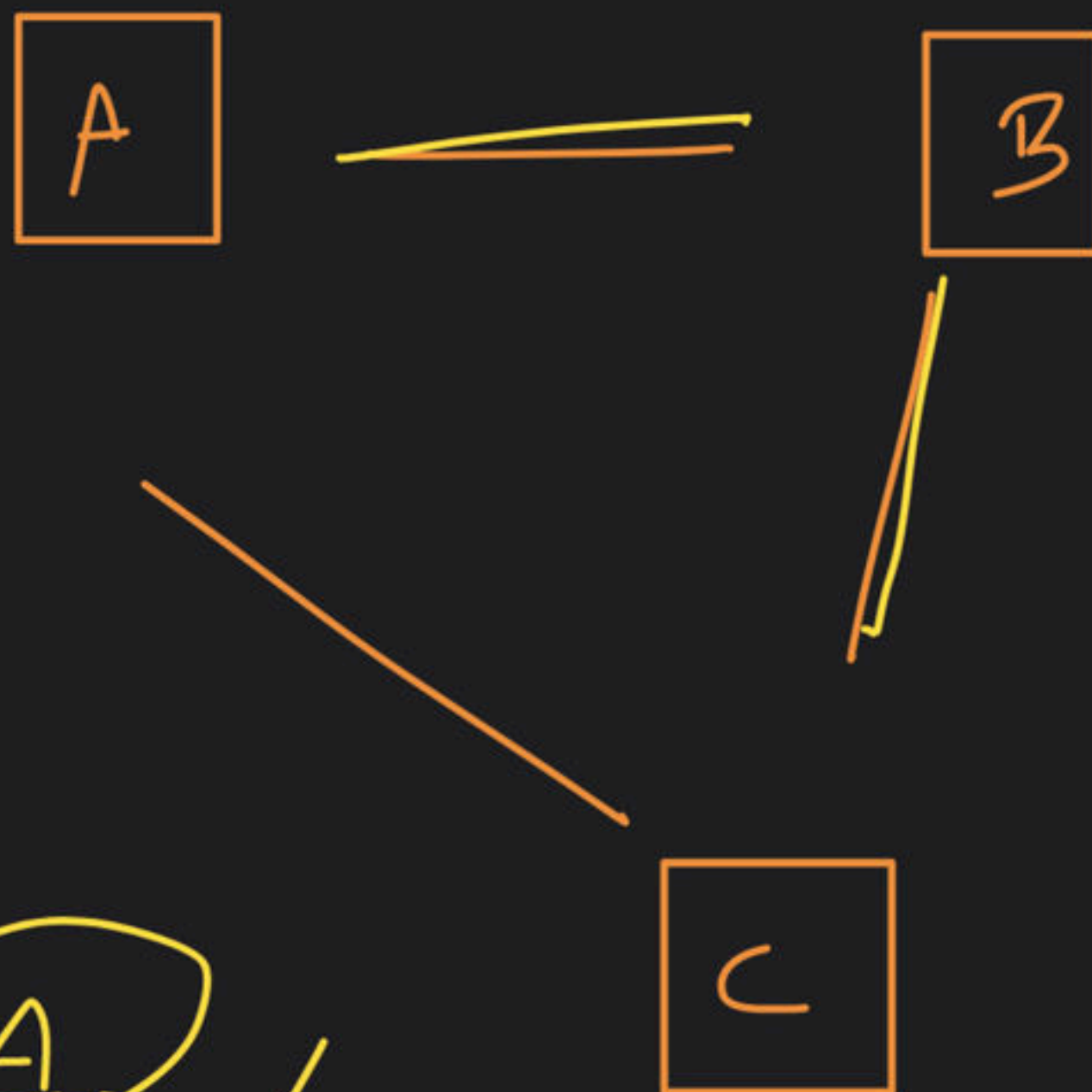
A → B → C → A

Cyclic graph



Acyclic graph

Path



A - B - C

A - B - C - A X

Components

connected component



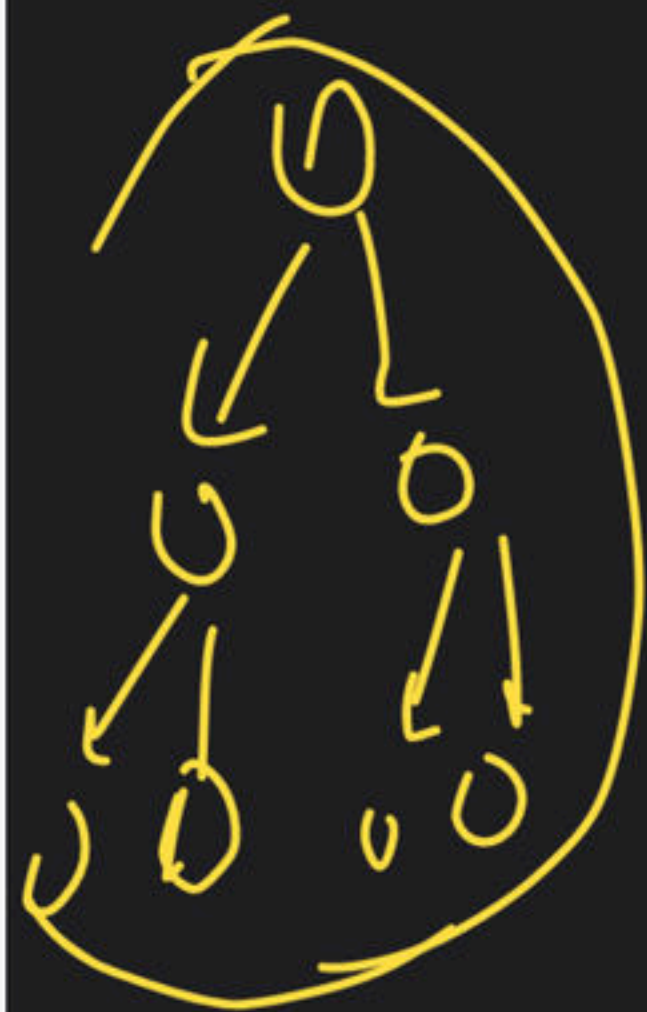
Component 1



Component 2



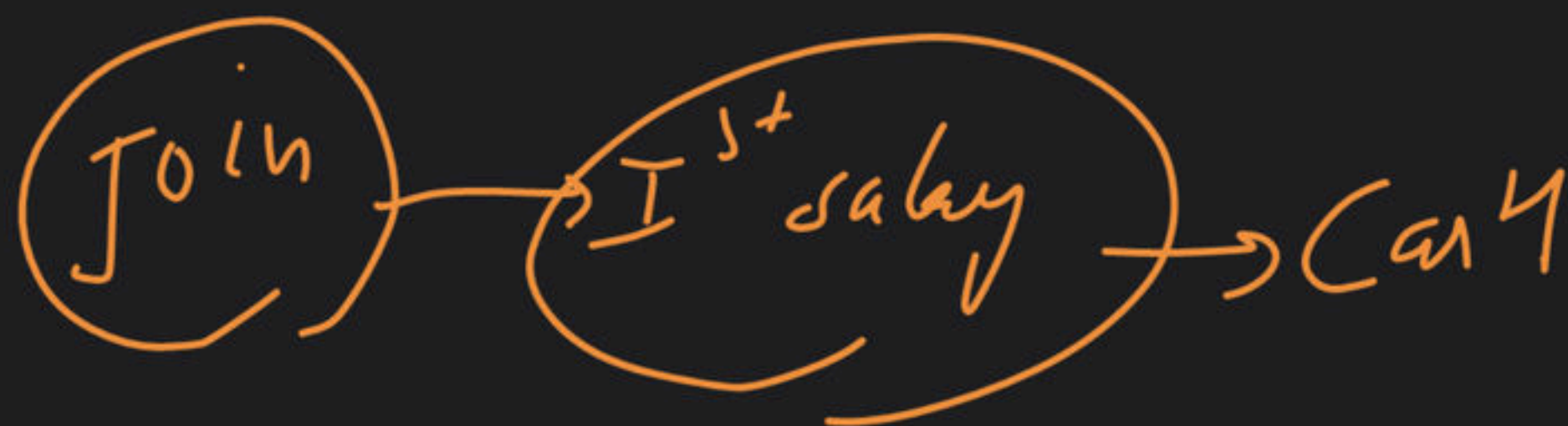
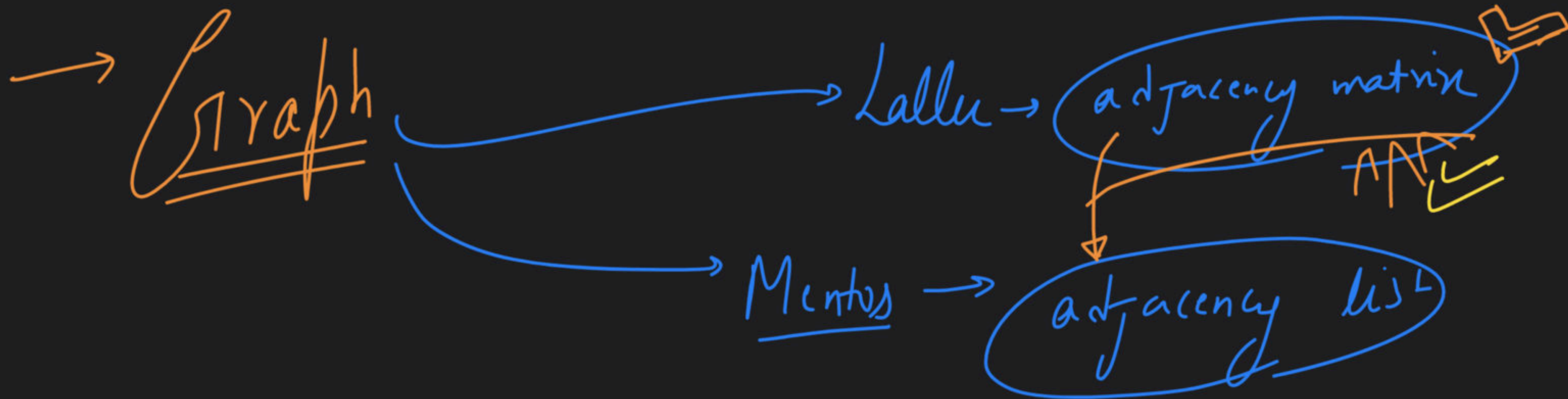
Component 3



graph g

disconnected components

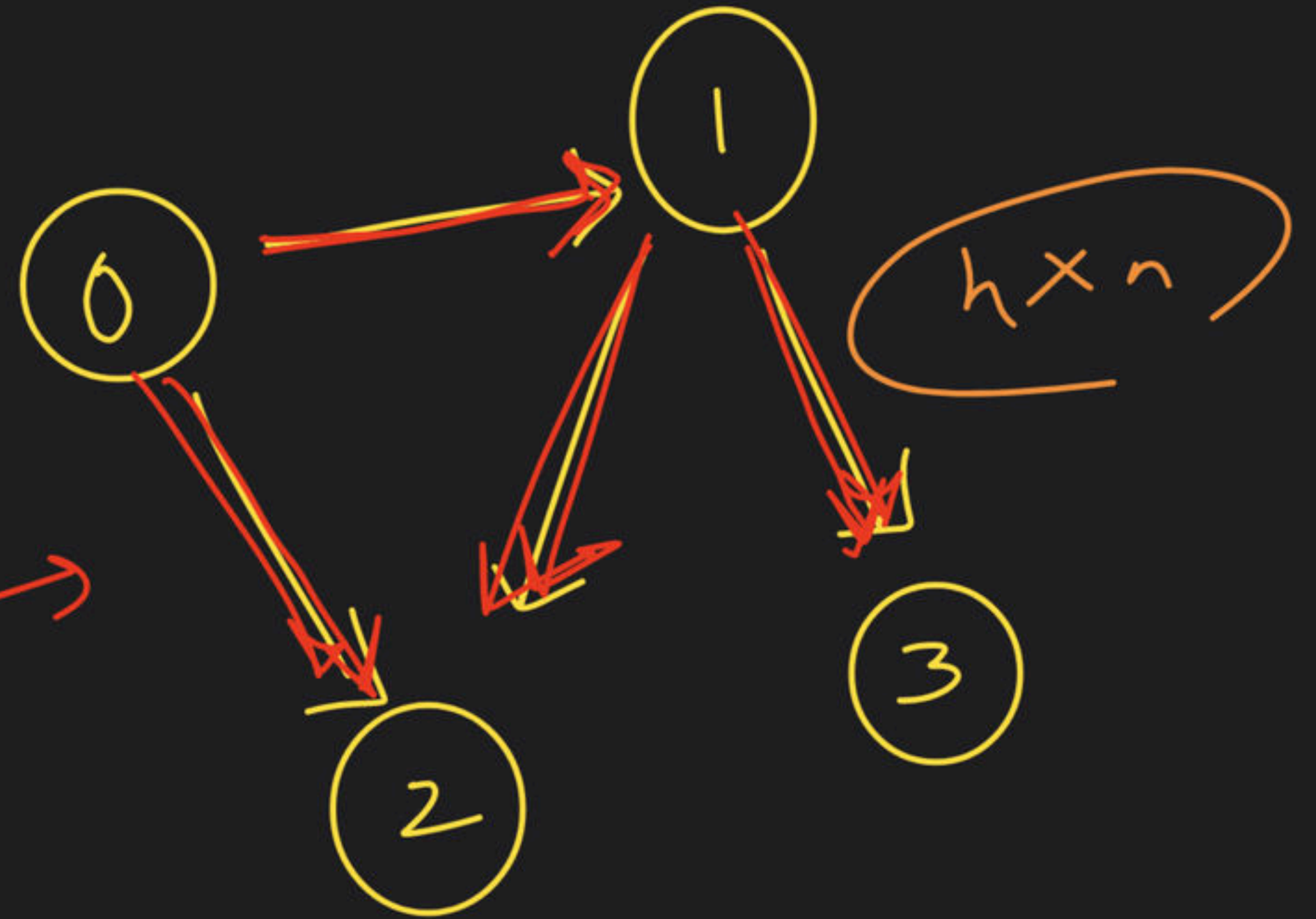
forest



Adjacency matrix → 2d array

h

	0	1	2	3
0	0	1	1	0
1	0	0	1	1
2	0	0	0	0
3	0	0	0	0

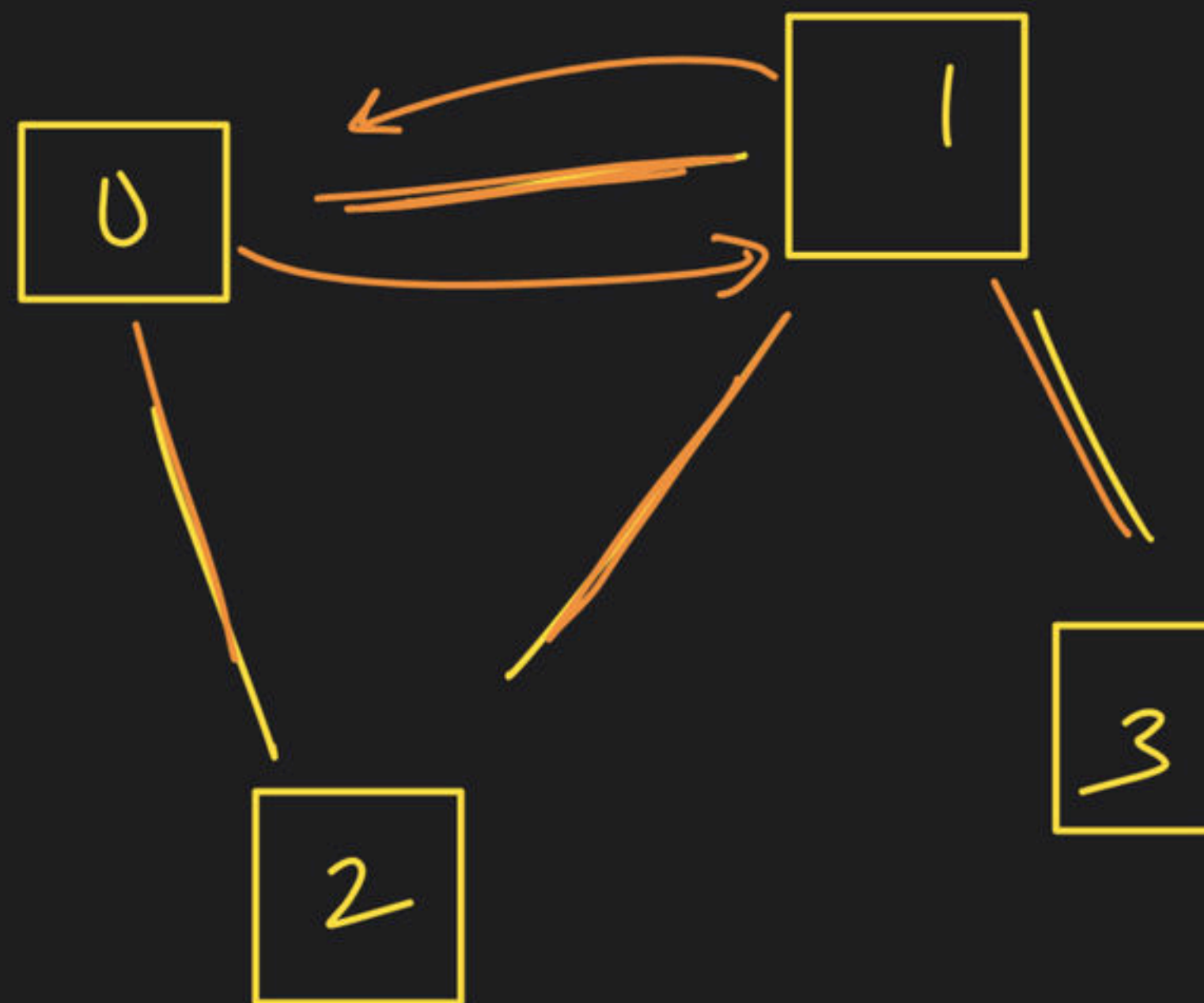


0 → ~~not connected~~ / edge

1 → ~~not connected~~ / connected

	0	1	2	3
0	0	1	1	0
1	1	0	1	1
2	1	1	0	0
3	0	1	0	0

0 — 1 $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ } 1 1



1-2 → $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$

a-M

$1 \rightarrow 2$

$[1][2] \rightarrow 1$

$1 \rightarrow 2$

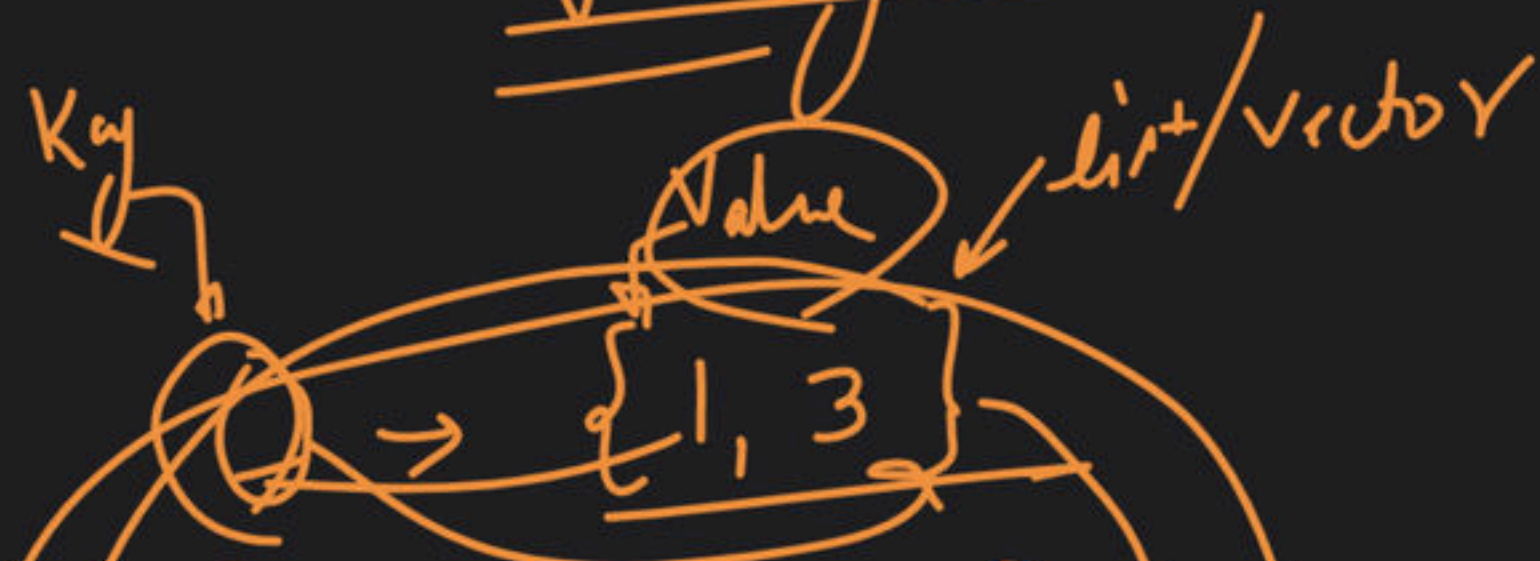
$[1][2] \rightarrow 4$

$[2][1]$

→ Graph → Implementation → Adjacency list

Padisiyon Ki list $\rightarrow$ graph

adjacency list



$1 \rightarrow \{2, 3\}$

$2 \rightarrow \{4\}$

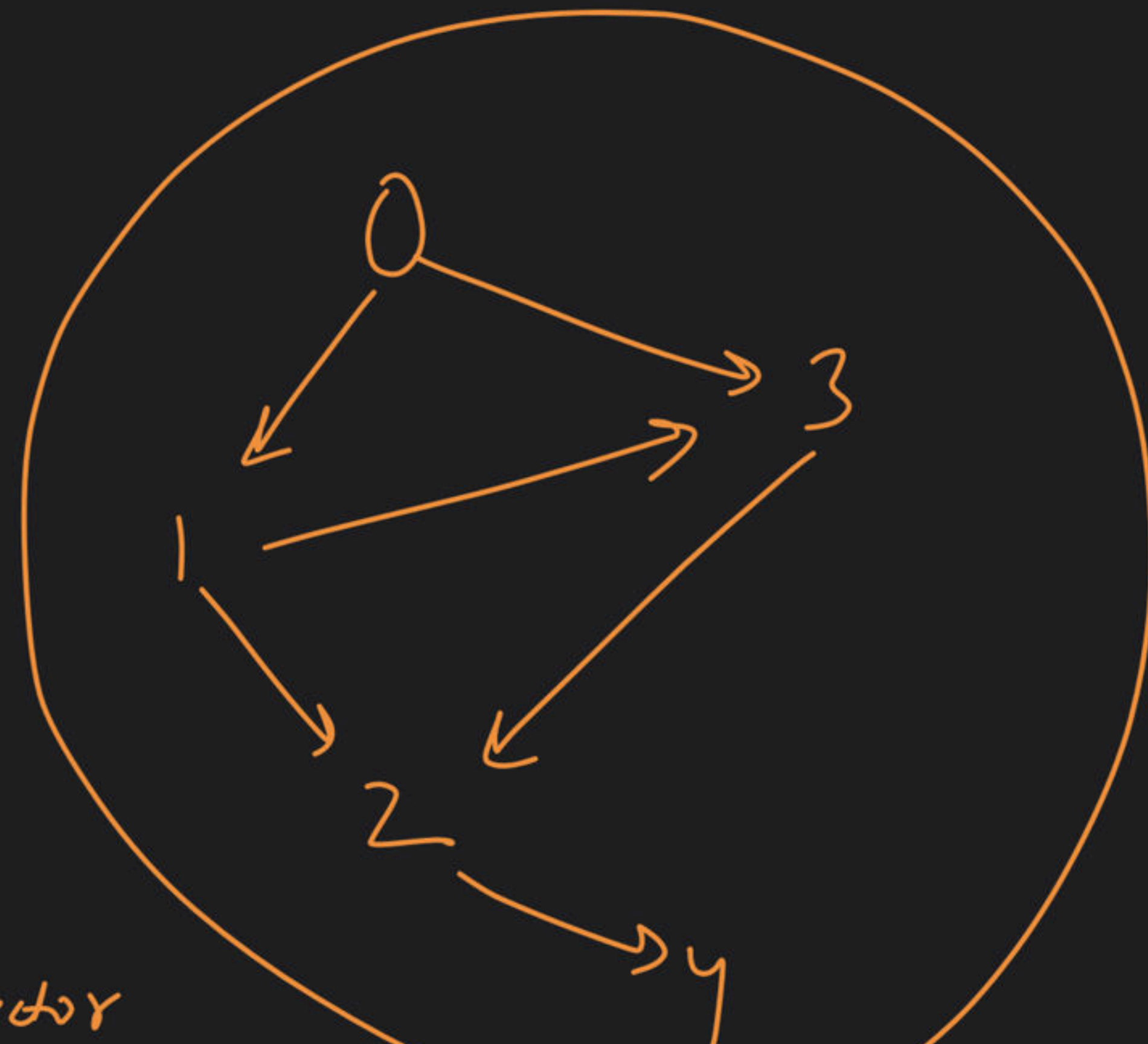
$3 \rightarrow \{2\}$

$4 \rightarrow \{\}$

Key $\rightarrow$ int

Value $\rightarrow$ list/vector

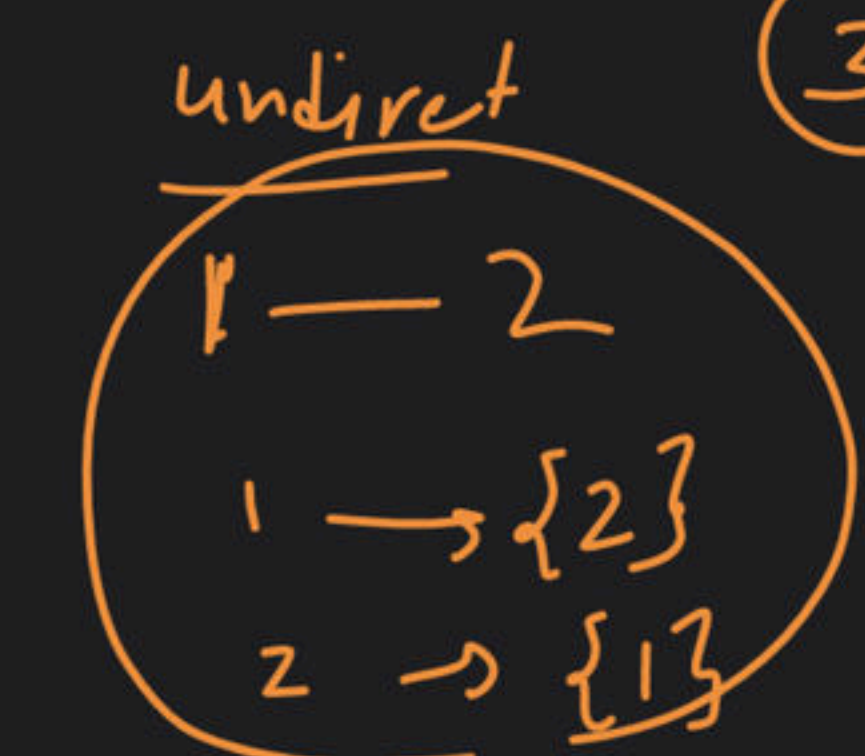
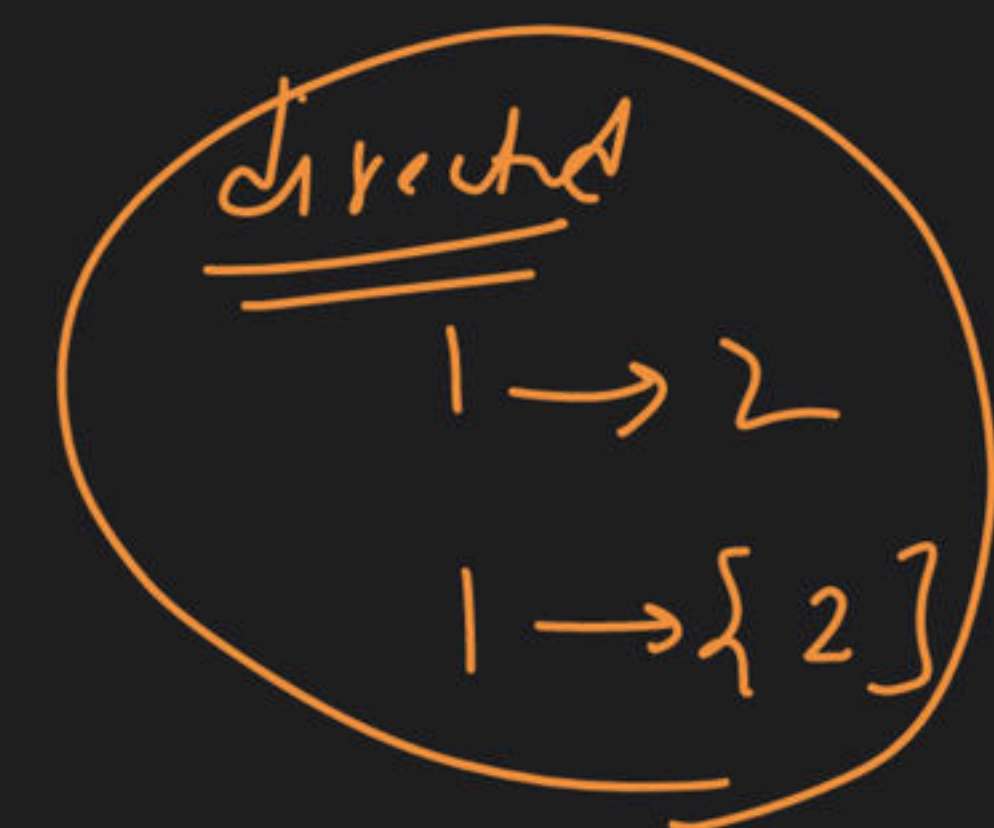
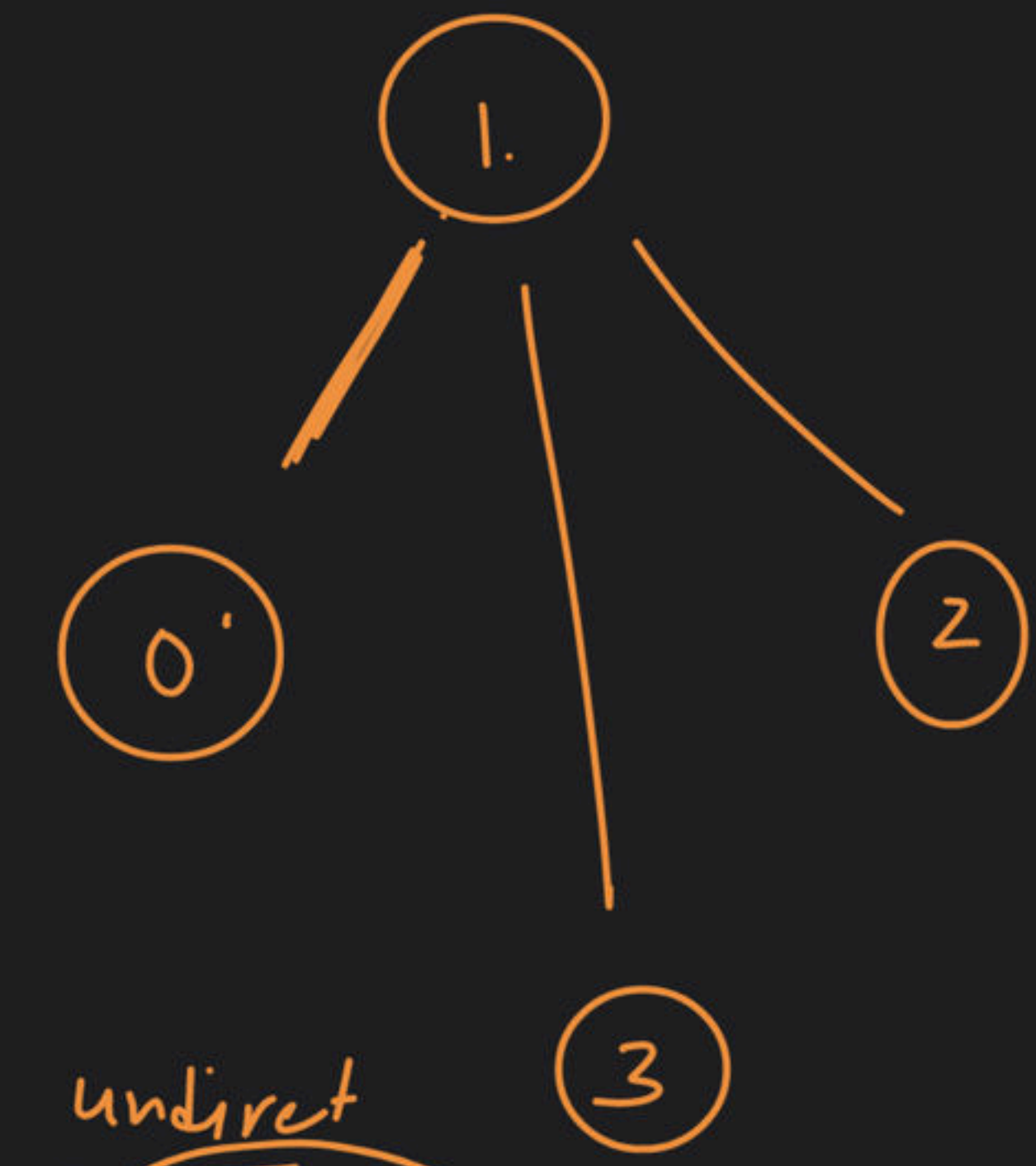
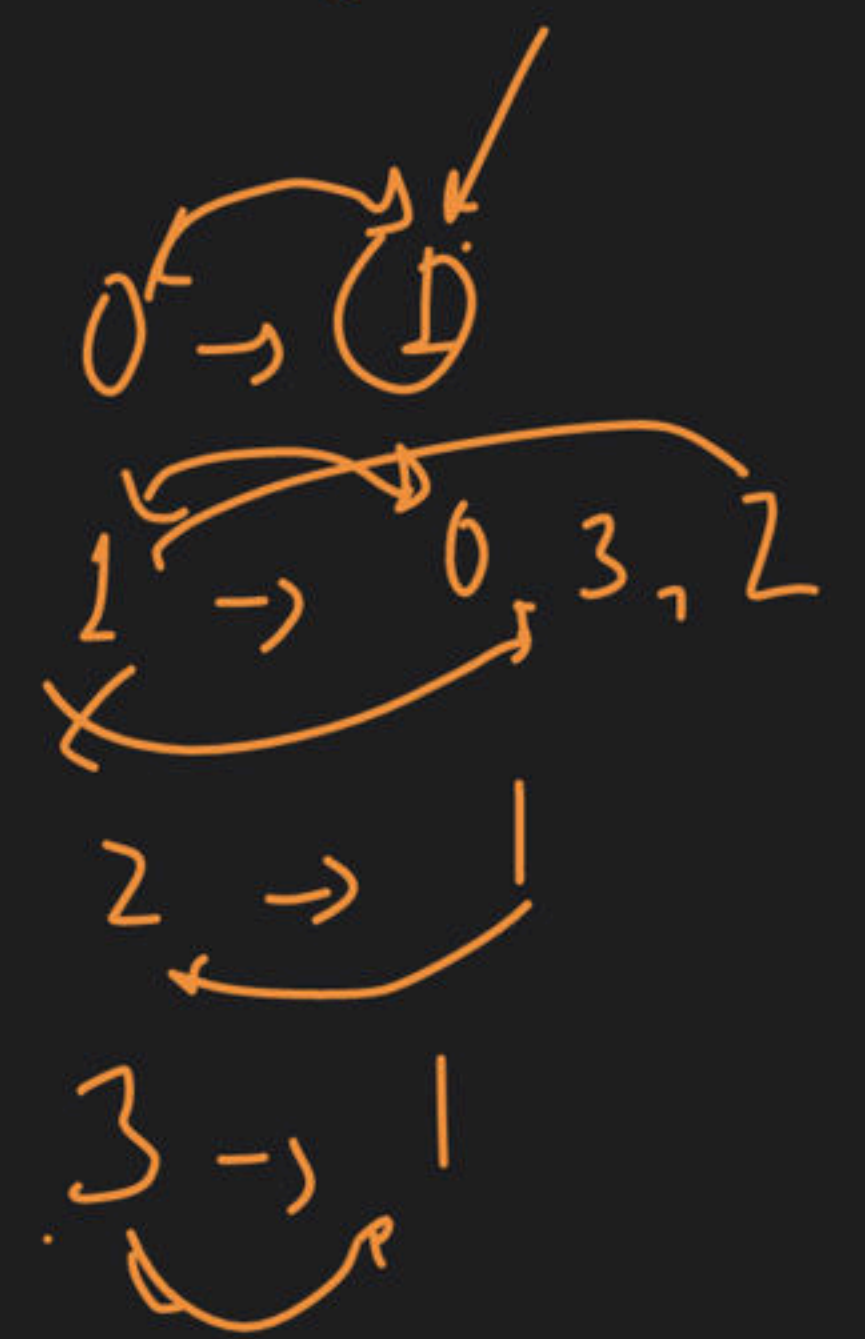
adj graph



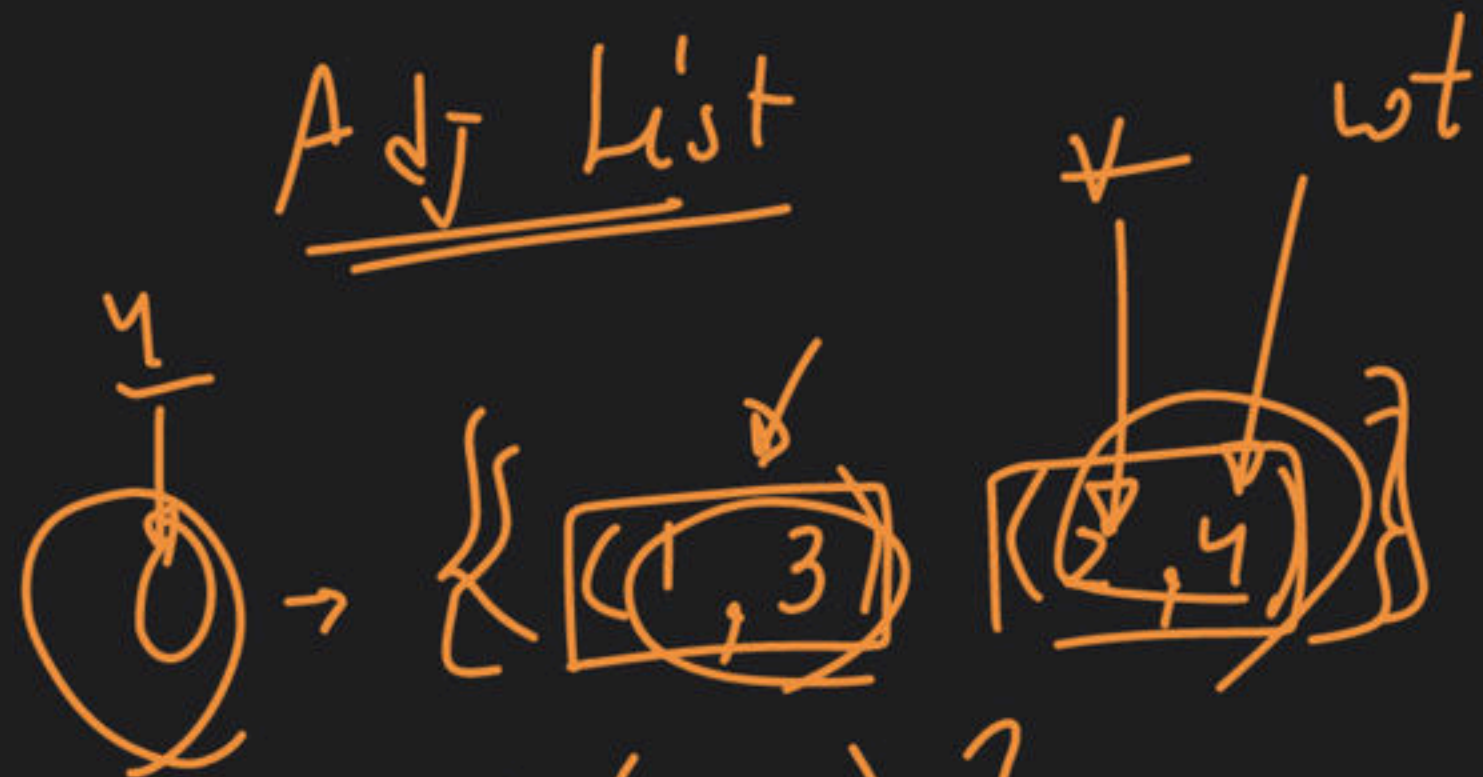
un-map $\langle$ int, list<int> $\rangle$

$\rightarrow$ graph

Adj. list



Adj List

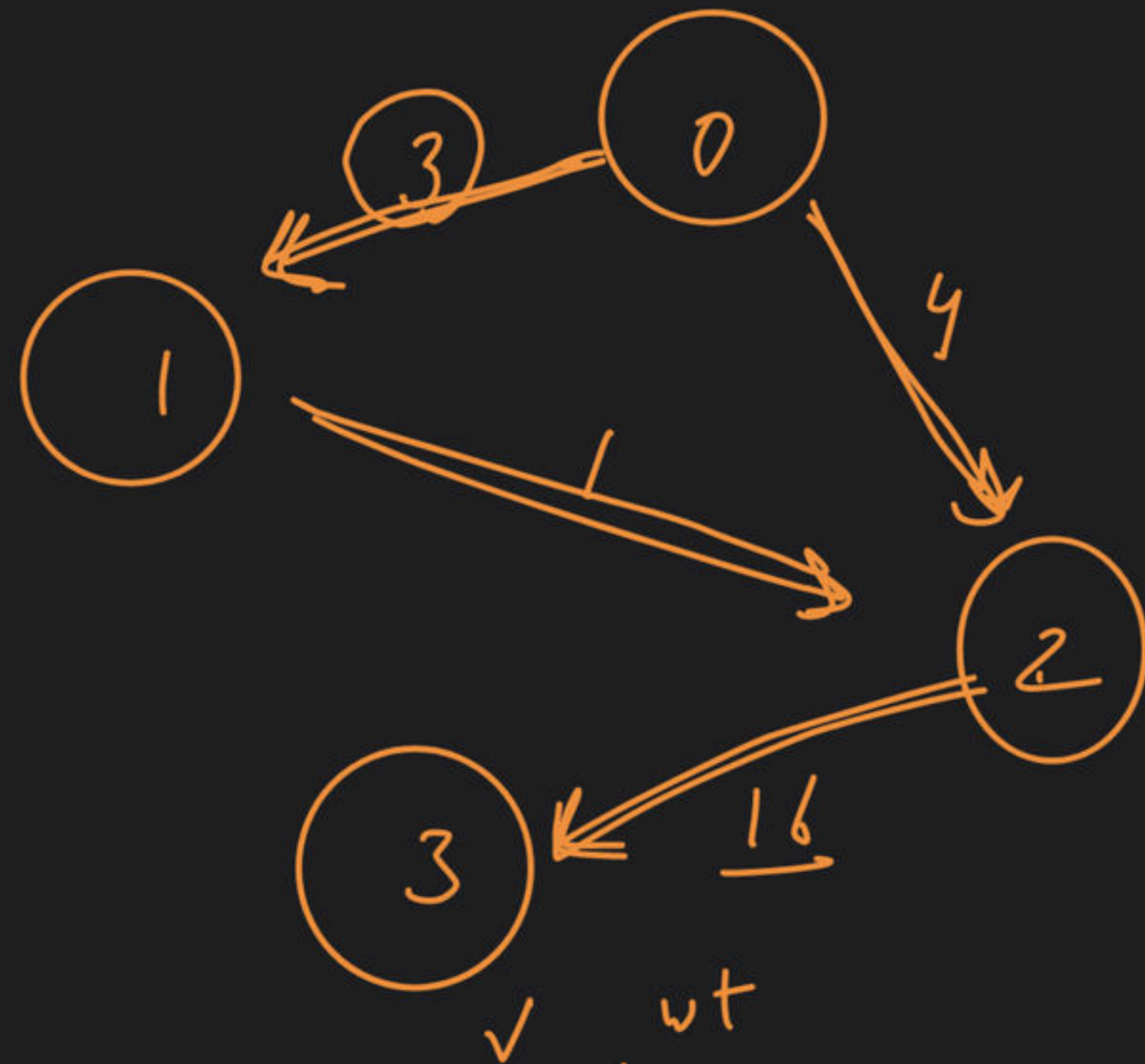


$1 \rightarrow \{(2, 1)\}$

$2 \rightarrow \{(3, 16)\}$

$3 \rightarrow \{\}$

unordered_map < int, list < pair < int, int > > adjList



H/W

(1) class Repeat $\rightarrow$ (2x)

(2) Code $\rightarrow$ T.C $\rightarrow$



$\searrow$
S.C $\rightarrow$



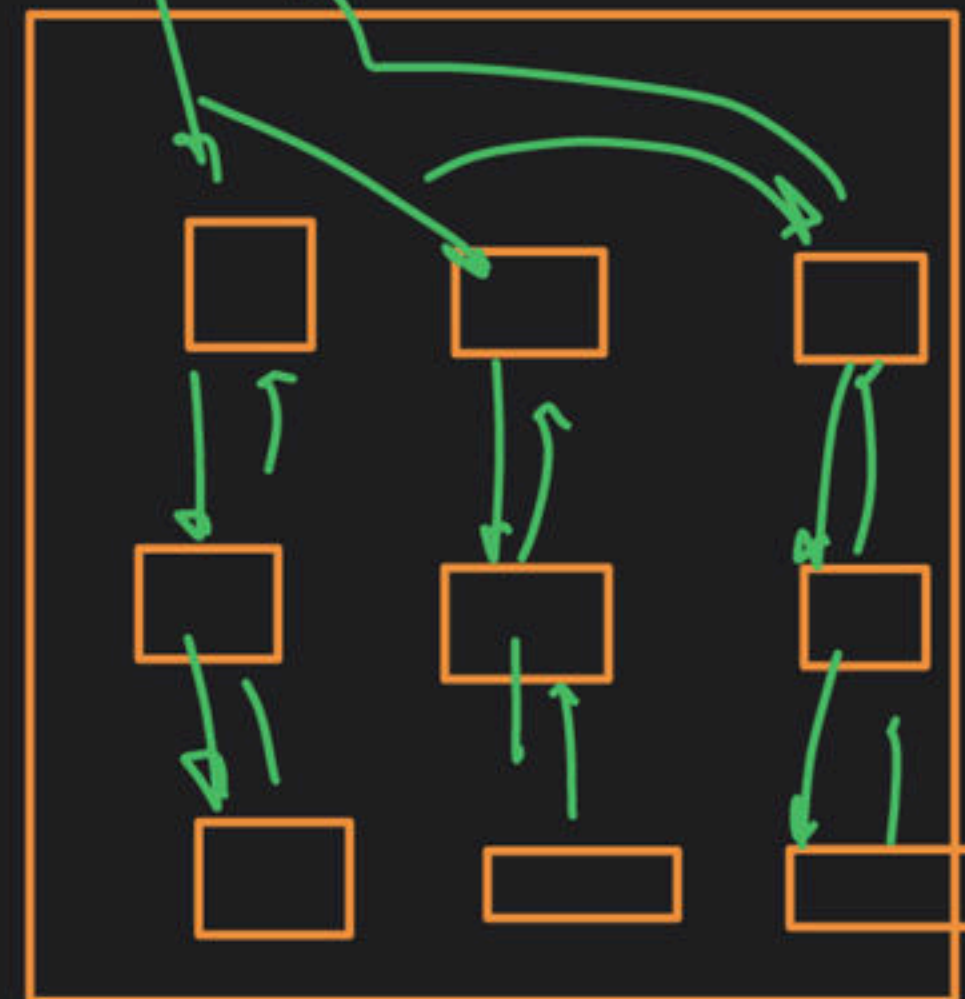
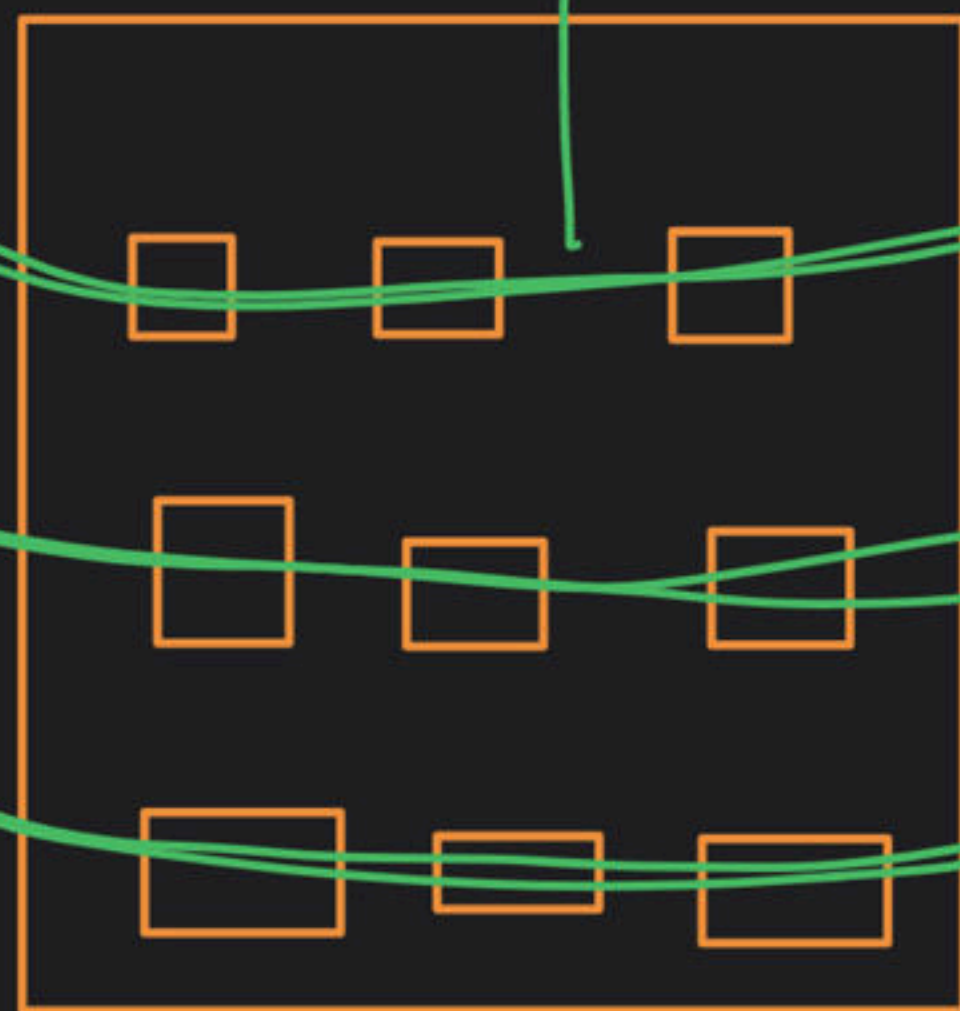
Traversal

BFS

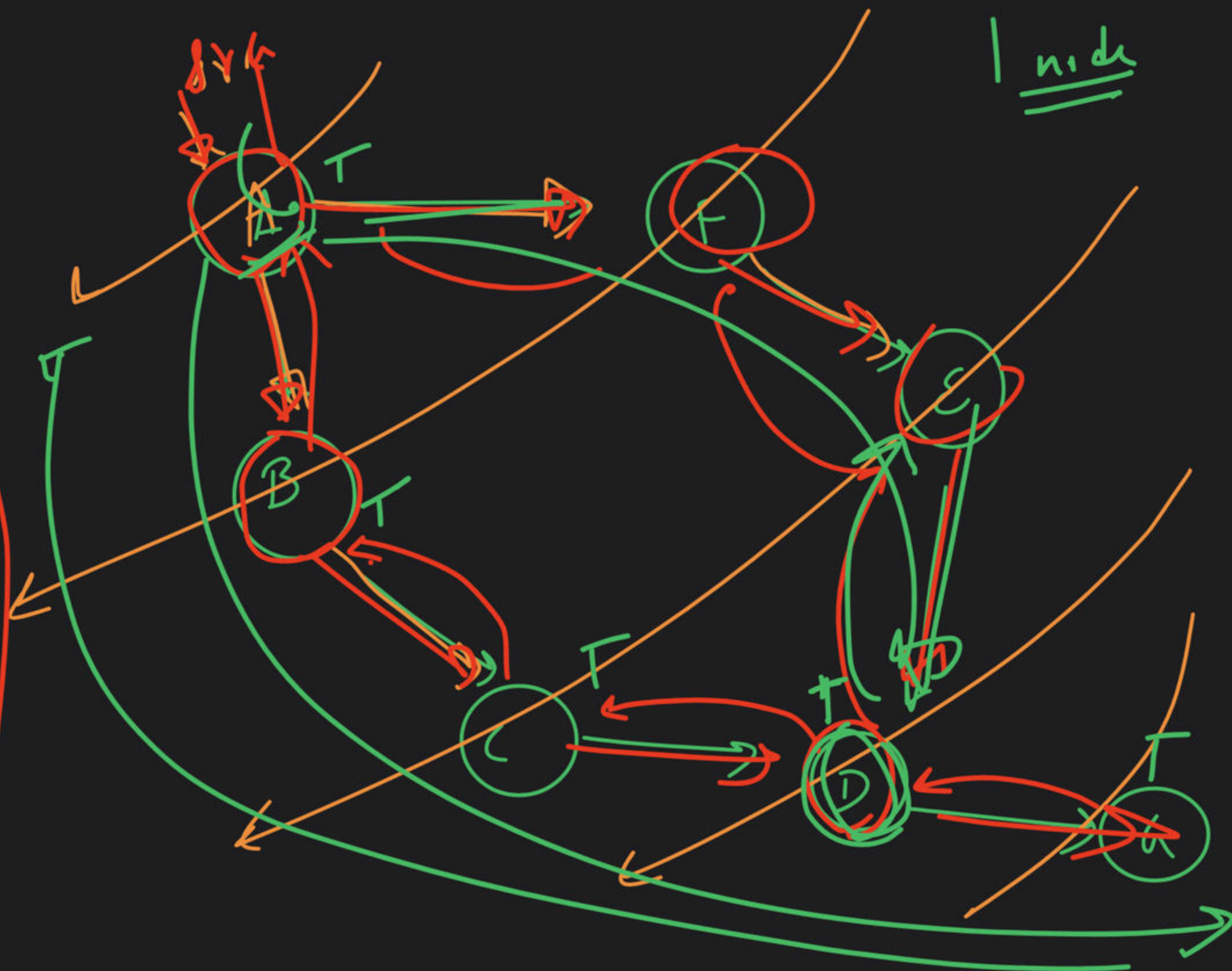
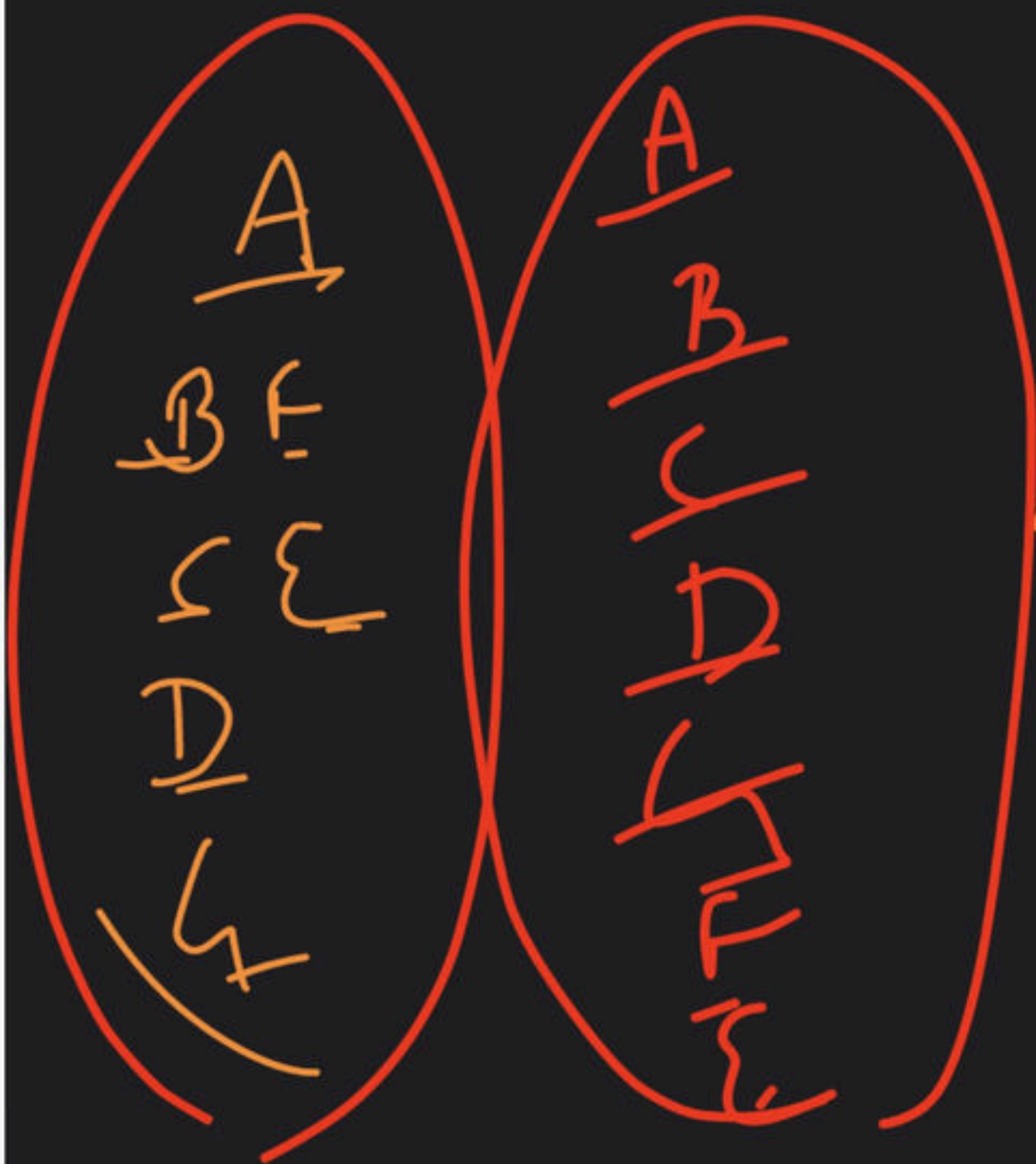
DFS

L.U.T

level
wise



BFS DFS



Queue

0 3 5 2 4 1 1

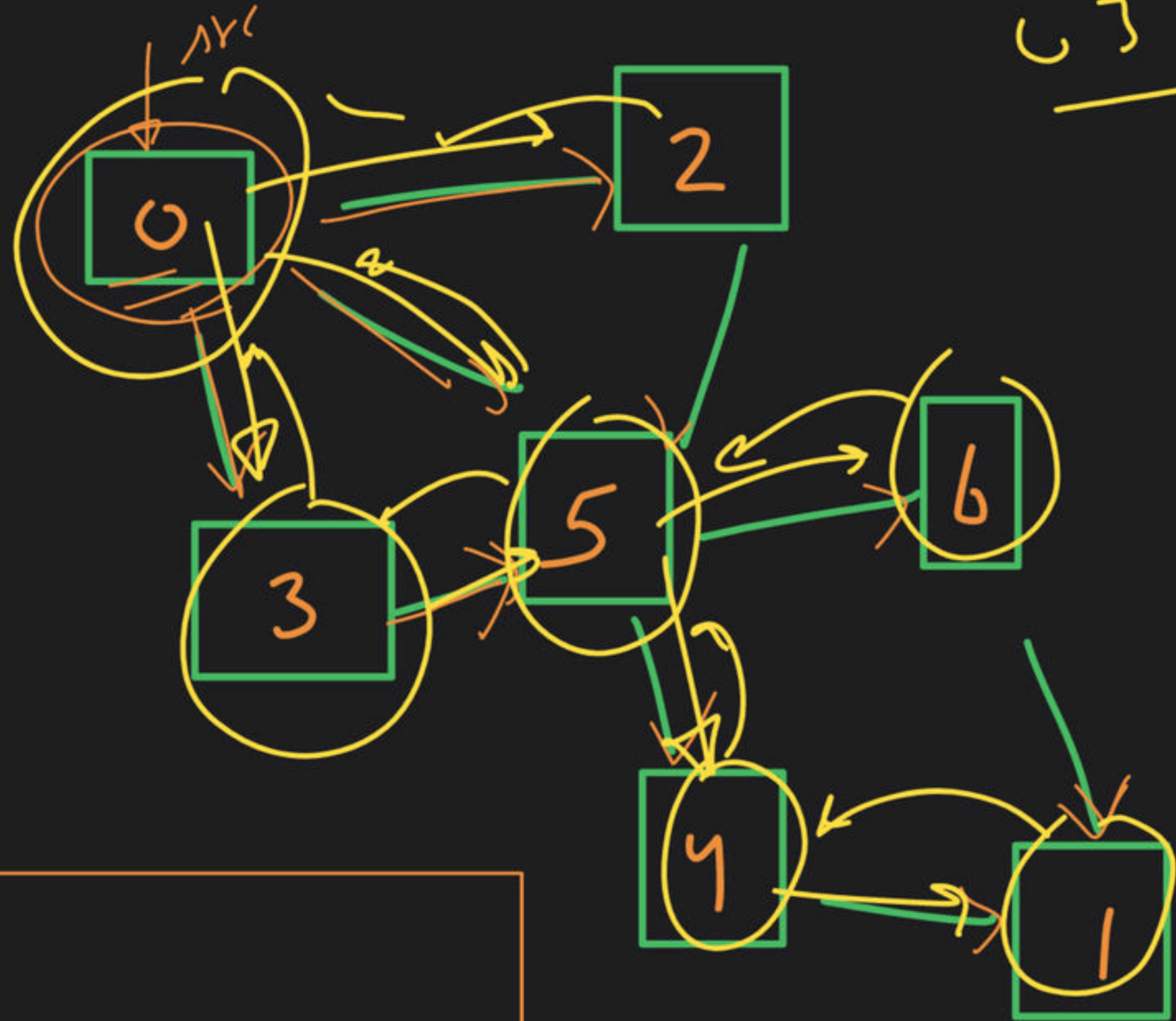
adj list

visited

0 → {3, 5, 2}
1 → {1}
2 → {5}
3 → {5}
4 → {1}
5 → {4, 6}
6 → {1}

0 → ~~X~~ T
1 → ~~X~~ T
2 → ~~X~~ T
3 → ~~X~~ T
4 → ~~X~~ T
5 → ~~X~~ T
6 → ~~X~~ T

~~0~~ | ~~3~~ | ~~5~~ | ~~2~~ | ~~4~~ | ~~1~~ | ~~1~~



0 3 5 4 | 6 2

① initial state $\xrightarrow{\text{src}} \text{vis}[\text{src}] = T$
② main logic $\rightarrow$ front
 $\rightarrow$ neighbor

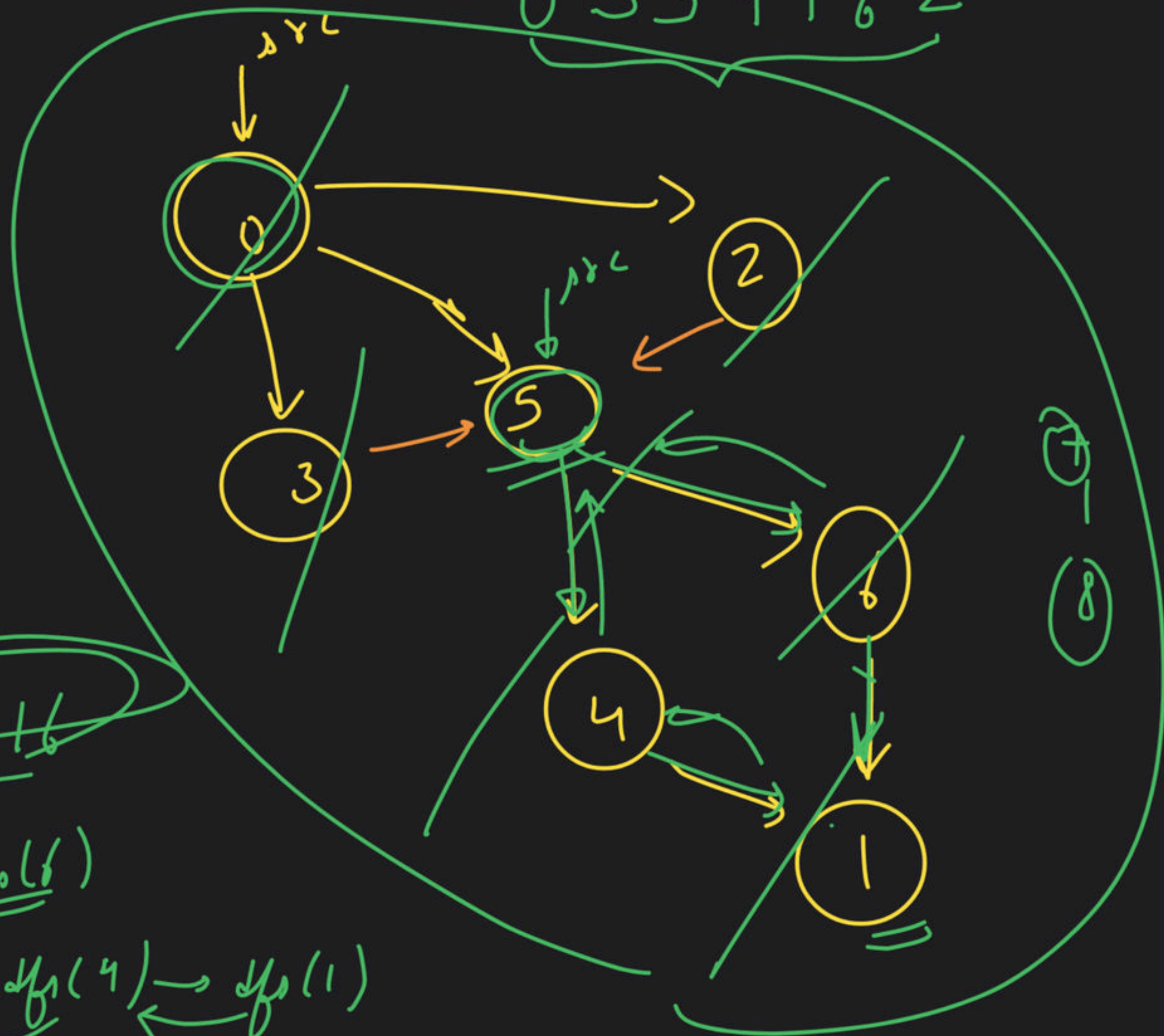
adj List

0 → {3, 5, 2}
 1 → { }
 2 → {5}
 3 → {5}
 4 → {1}
 5 → {4, 6}
 6 → { }

visited

0 → ~~F~~ T
 1 → ~~F~~ T
 2 → ~~F~~ T
 3 → ~~F~~ T
 4 → ~~F~~ T
 5 → ~~F~~ T
 6 → ~~F~~ T

0 3 5 4 1 6 2



dfs(5) → 5 4 1 6

dfs(0) → dfs(3) → dfs(5) → dfs(4) → dfs(1)

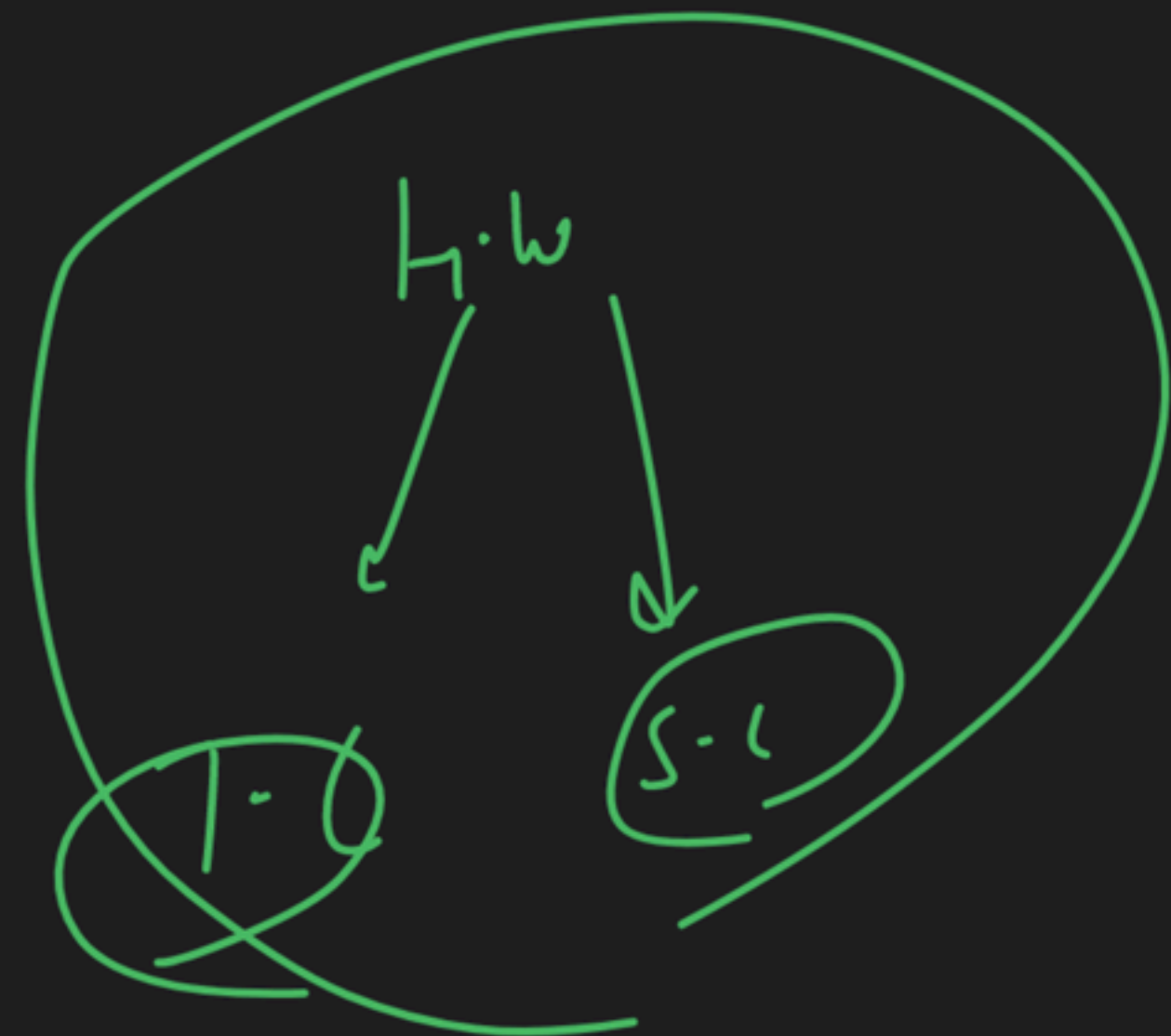
0 1 2 1

0 → 1
1 → 0

visited



2 ← ~~0~~ ~~1~~ ~~0~~ ~~1~~
~~0~~

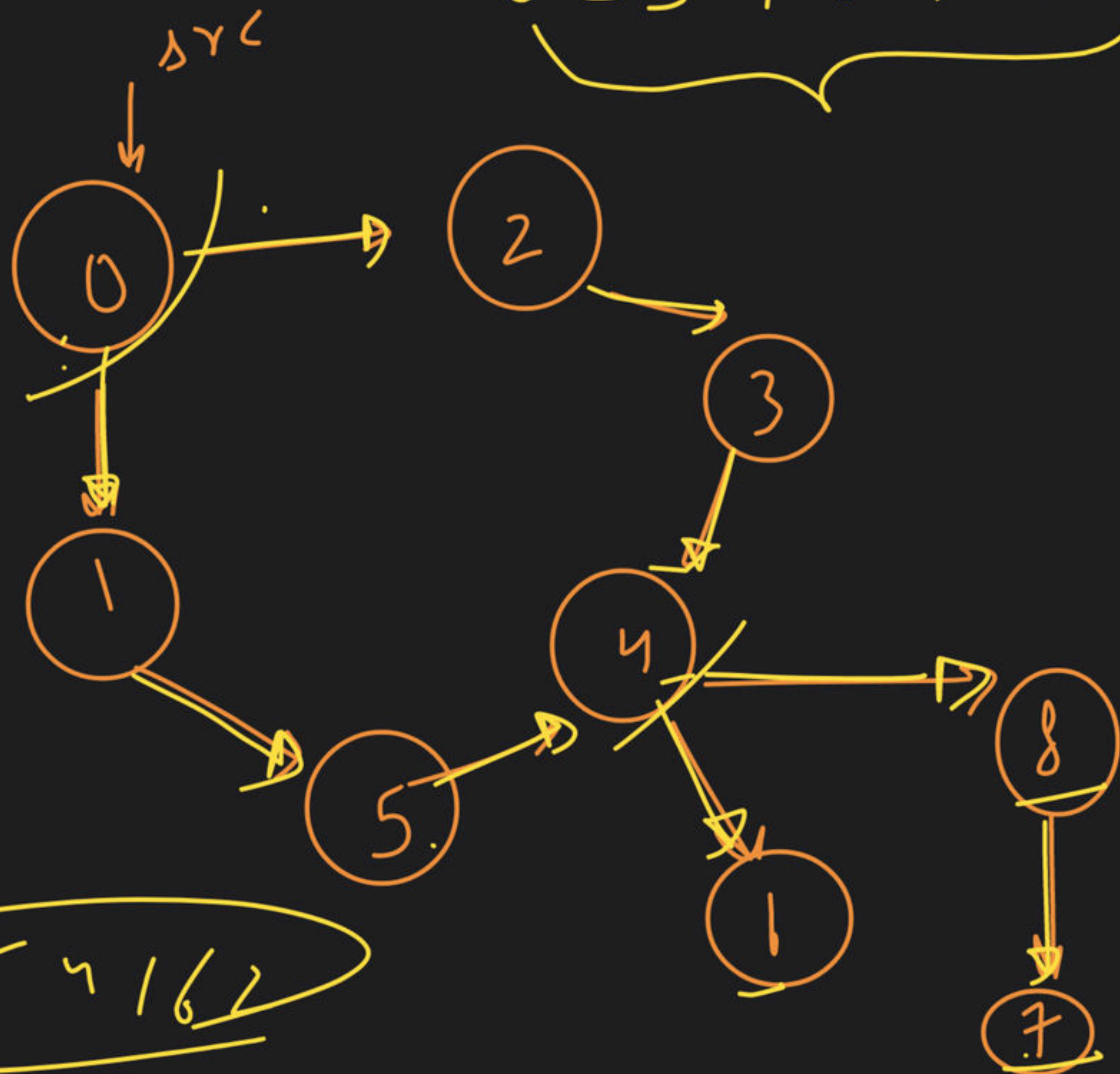
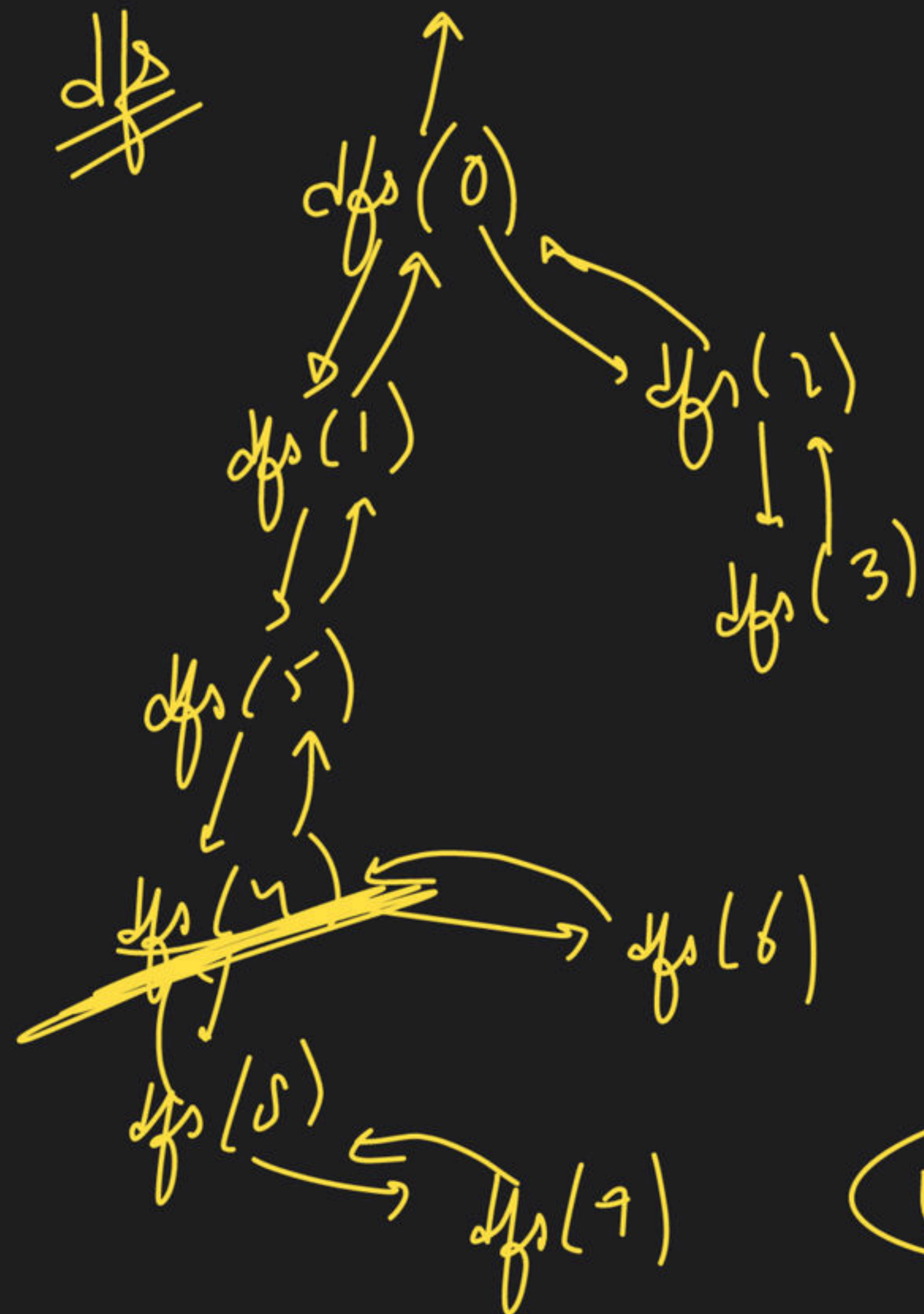




Kal
class
hai



~~dfs~~



0 3 5 4 1 6 2

















